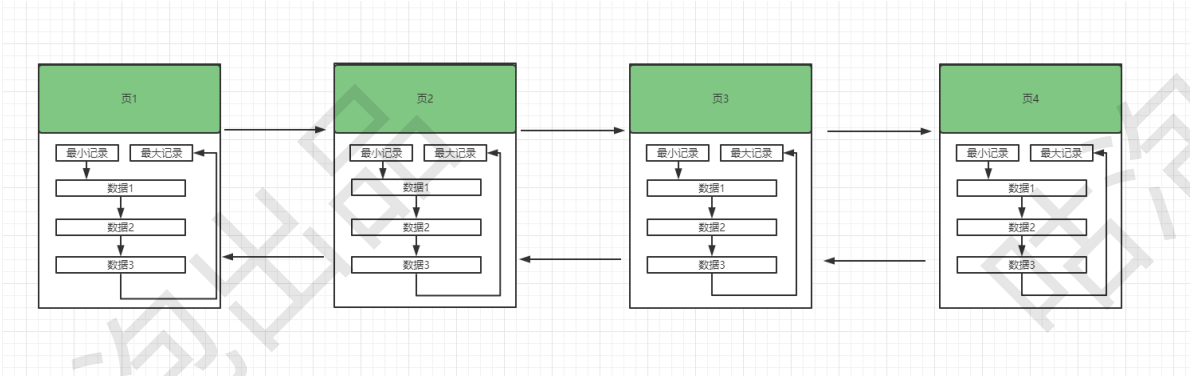
Mysql索引剖析

从Page页讲起



page里面每条数据都会有1个标记，叫做record_type

0：普通用户记录

1：目录记录

2：最小记录

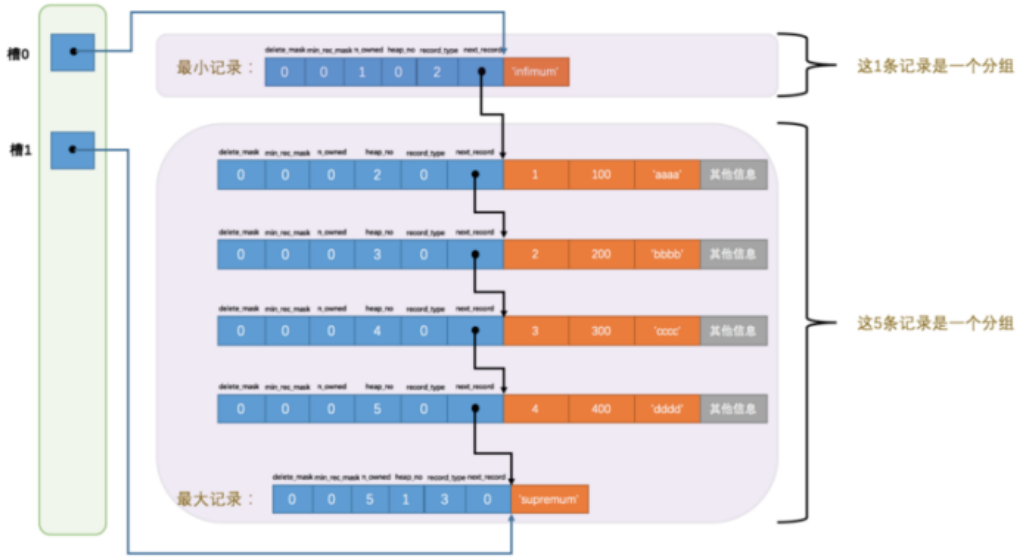
3：最大记录

这个是我们page页，我们每个page都会存放数据，现在如果按照现在这样的逻辑，我们假如要查1条数据要怎么查?

如果在同一页中

1.如果是主键，那么去找，相对快点，会通过槽的概念使用二分法去快速定位

咕泡出品



2.因为页的数据分槽是根据主键，所以，如果是根据非主键索引，只能从最小的一个一个遍历链表

如果在不同的页中

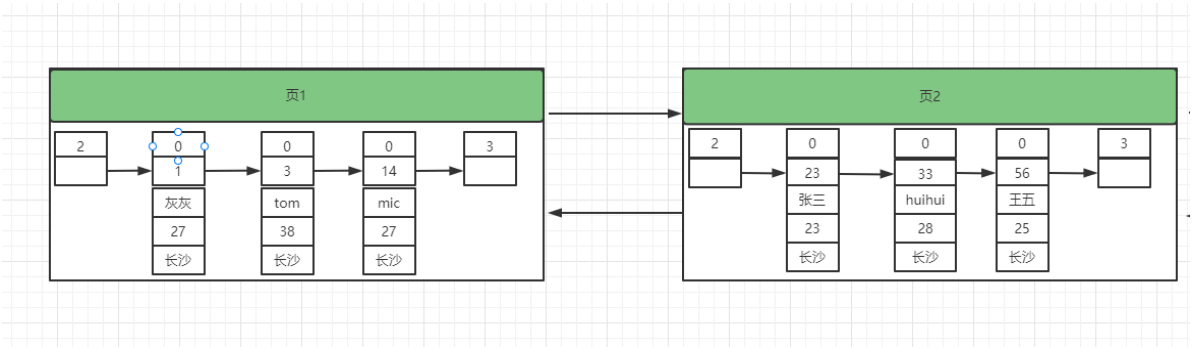
由于我们并不能快速的定位到记录所在的页，所以只能从第一个页沿着双向链表一直往下找，在每一个页 中根据我们刚刚在同一个页的查找方式去查找指定的记录

主键索引

https://www.processon.com/view/link/61a0f8ff0e3e743d103f3968

因为慢，所以面试经常问的索引就这么出来了！！我们接下来看下索引树到底是怎么出来的

假如现在我以现在表里的数据为例，假如我一个page页只能放3条数据，那么页中的数据是这样的。



现在如果我要再加一条数据，那么是不是得开启一个新页！

并且页的数据必须满足一个条件：

下一个数据页中用户记录的主键值必须大于上一个页中用户记录的主键值

那么假如我再加一条数据主键为2的。

首先，必须要生成一个新页

1.把56的数据移到新页

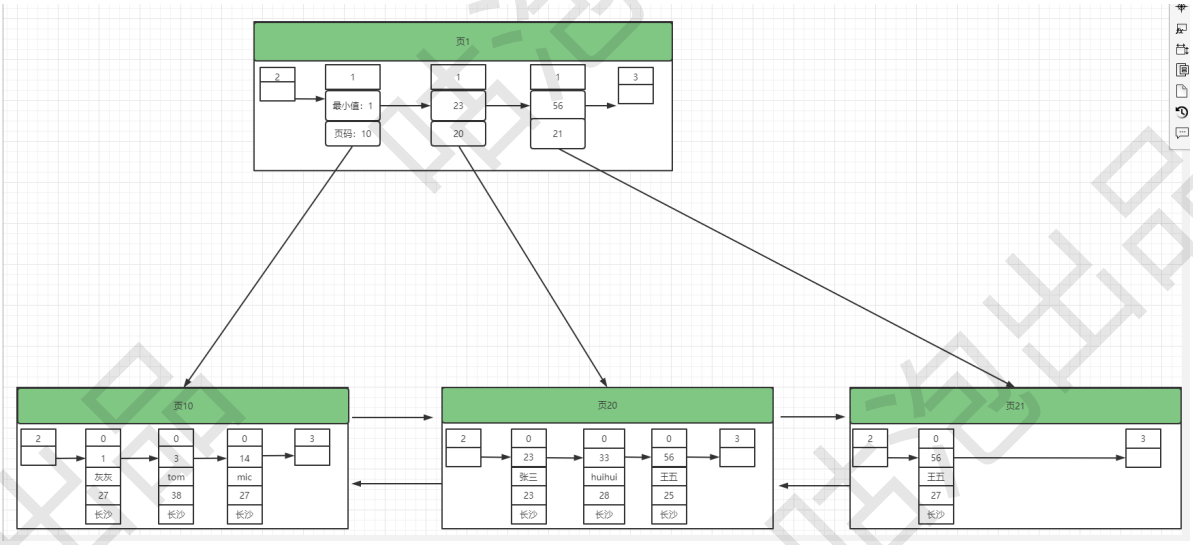
2.讲ID为2的放到第二个页

3.发现第二个页的2还是小于第一个页

4.把14移到第二个页，讲2添加到第一个页

这个过程就叫做页分裂！！

面试题：为什么主键ID最好是能趋势递增？



我们看下满足了

1.实际用户记录其实都存放在B+树的最底层的节点上，那么所有的sql查询查到真实数据的路径一致的，所以稳定性更高

2.数据只在根节点，那么

如果B+树只有1层，也就是只有1个用于存放用户记录的节点，最多能存放100条记录。

如果B+树有2层，最多能存放1000×100=100000条记录。

如果B+树有3层，最多能存放1000×1000×100=100000000条记录。

如果B+树有4层，最多能存放1000×1000×1000×100=100000000000条记录。哇咔咔～这么多的记录！！！

面试题

mysql innoDb索引为什么用B+树?

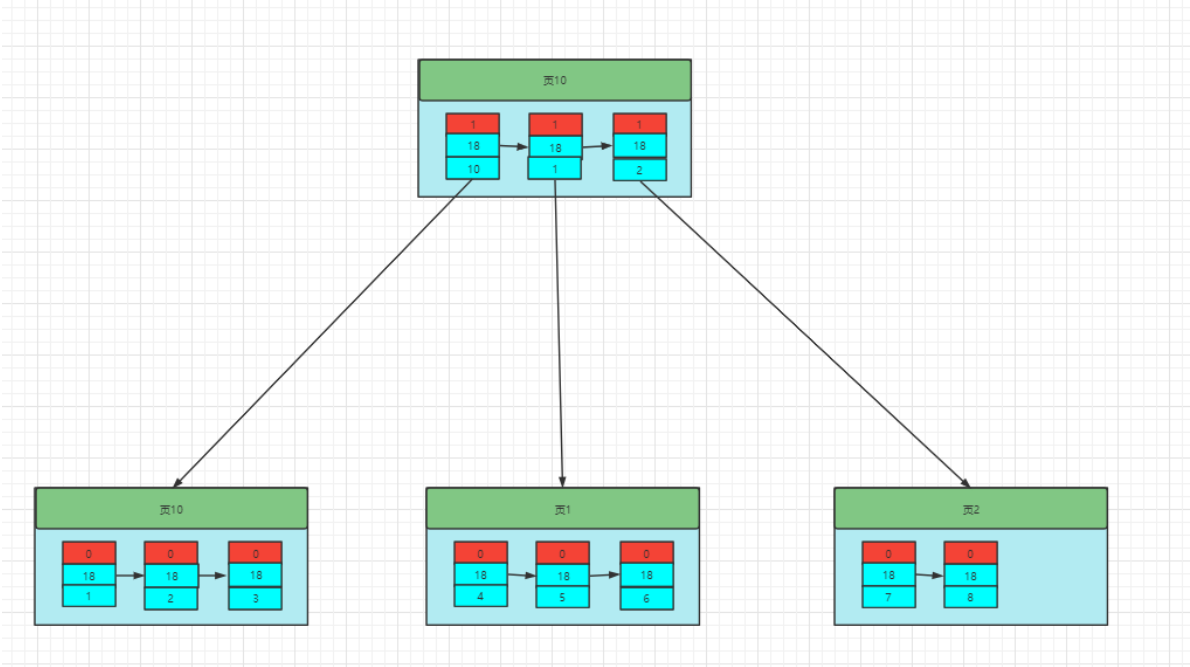
mysql 大字段为什么要拆分?

二级索引

https://www.processon.com/view/link/61a0f8f07d9c083e98b3122e

为什么非叶子节点要存聚集索引的主键ID?

我们先来看个场景：



该场景是我的目录页只有二级索引值与页号，并且我的二级索引3页都是18！！

现在，我要去添加一条二级索引为18，id为10的的数据！！

按照我们之前的知识，添加数据时我必须先找到应该添加到哪个位置？

现在我们发现我这个18不知道放哪里了？因为3个页的数据都是18.。。。。。但是如果我目录页有主键值的话，我就可以判断出放在最后一页。

得出一个结论：除了页号以外，同一节点其他几个组成元素必须是唯一的

跟这个原因相似的知识点问题有：

1.为什么聚集索引需要是唯一索引？因为如果不唯一，我的聚集索引插入相同数据时，不知道位置在哪！！跟上述结论违背；还有索引即数据，保证数据的唯一性

2.为什么离散度比较低的字段不适合建立索引？因为离散度越低，那么去查询响应位置的次数也就越多，耗时也就越多

每个索引都会有个索引树！刚才我们讲了离散度低的不适合建索引，那么面试题又来了！

那么索引是不是越多越好，会带来什么问题？

1.空间上，索引是为了快，会存有额外的数据，所以，需要牺牲一定的空间，每个索引都会有个索引树，如果索引过多，那么占用的空间也就越大

2.时间上，虽然索引，如果走到索引，查询会快很多，但是，添加、修改、删除数据的时候，都需要对索引树进行修改！！操作数据的速度会变慢很多！！

联合索引

假如，我们以age、name建立一个联合索引

https://www.processon.com/view/link/61a0f8e363768975f850c191

一条sql是怎么走索引的

建立teacher_age与teacher_name的联合索引

我们来分析下这条语句怎么走

```
SELECT * from gp_teacher WHERE teacher_age=18
```

前面的什么语法解析、词法解析我们就不讲了，我们从到innoDB开始

1.我们先判断能不能走索引？ 因为有联合索引teacher_age 跟teacher_name这样的联合索引树，所以肯定能走到联合索引树

2.我们从索引树里面能查出 teacher_no、teacher_age、teacher_name。因为这些数据在索引树都有的

3.但是，我们的sql是*,所以还有个字段teacher_address查不到，我们需要去聚集索引查，那么这个步骤就是回表。

回表：通过非聚集索引查询时，我们在非聚集树查到聚集索引，然后再从聚集索引树中去查询真实数据

4.那么如果我只查

```
SELECT teacher_no,teacher_name,teacher_age from gp_teacher
WHERE teacher_age=18
```

这样，所有的数据都在我的二级索引数里，不需要再去聚集索引拿，也就是不需要回表！

这个场景就是覆盖索引：

如果select的数据列只用从索引中就能够取得，不必从数据区中读取，这时候使用的索引就叫做覆盖索引

为什么mysql优化的时候，有一个点，一般不查询*,而是先查出ID，然后通过再查询数据！！

其实目的就很简单，减少回表！！

索引下推：

Mysql5.6之后新加功能，减少回表数量

```
SELECT * from gp_teacher WHERE teacher_age=18 and
teacher_name LIKE '%tom%'
```

这个sql是能走到索引的，但是只有teacher_age能走到索引，因为teacher_name以%开头，teacher_name就是不知道什么开头，所以teacher_name用不到索引

假如，teacher_age=18有300条数据，包含tom的有100条

在5.6之前

在二级索引中查到teacher_age=18,查到300条数据，300条数据全部回表，然后给到service层过滤包含tom的，得到100.

5.6之后

在二级索引中查到teacher_age=18,查到300条数据，然后在存储引擎中直接过滤掉没有tom的200条，然后回表只需要回表100条。返回给server

咕泡出品

索引条件下推，其实就是把之前server层过滤，下推到存储引擎！！